Applying Reinforcement Learning to 2D Precision Platformer Games

First Author
國立成功大學
P76151466
楊乃諺

Second Author
國立成功大學
P76151123
王家騏

Abstract

Reinforcement Learning (RL) has demonstrated impressive performance in learning game-playing agents, particularly in environments such as Atari games. Deep reinforcement learning methods, such as Deep Q-Network (DQN) and Proximal Policy Optimization (PPO), allow an agent to learn control policies directly through interaction with an environment rather than relying on manually designed rules.

*In this project, we investigate the application of reinforcement learning to **2D precision platformer games**, where successful gameplay requires accurate movement, timing, jumping, and continuous decision-making. Unlike conventional platform games, precision platformers require the agent to carefully control its horizontal velocity and jumping trajectory, making them a challenging environment for reinforcement learning.*

*We plan to construct a reinforcement learning environment based on a simplified 2D platformer and train an agent using **Proximal Policy Optimization (PPO)**. The agent will receive visual observations or game-state information and select actions such as moving left, moving right, jumping, or performing no action. We will investigate different state representations and reward functions to determine how they affect learning efficiency and gameplay performance.*

The expected outcome is an agent capable of learning basic movement and platform navigation without manually programming a complete movement strategy. We will evaluate the agent using metrics such as completion rate, maximum progress, number of deaths, reward, and training efficiency. The project will also investigate the difficulties caused by sparse rewards and precise timing requirements in 2D platformer environments.

1. Introduction

1.1. Motivation

Reinforcement Learning is a machine learning framework in which an agent learns how to make decisions by interacting with an environment. Instead of being explicitly programmed with a fixed strategy, the agent learns through trial and error by receiving rewards or penalties after taking actions.

Video games provide useful environments for studying reinforcement learning because they have clearly defined states, actions, objectives, and feedback. Previous research has demonstrated that deep reinforcement learning can learn effective game-playing policies directly from visual observations. In particular, the Deep Q-Network proposed by Mnih et al [1] demonstrated that an agent could learn to play Atari games using only screen pixels and game scores as input.

OpenAI's Gym Retro [4] further demonstrated the potential of using video games as reinforcement learning environments. Gym Retro provides interfaces for a large collection of games and allows researchers to define starting states, reward functions, and termination conditions for reinforcement learning experiments.

However, many conventional game benchmarks do not fully represent the difficulty of precision movement. In a precision platformer, simply moving toward the goal is insufficient. The agent must learn when to accelerate, when to stop, how to control its position while airborne, and how to land on relatively small platforms.

Therefore, this project focuses on the following research questions:

Can a reinforcement learning agent learn precise movement and platform navigation in a 2D precision platformer without explicitly programming the movement strategy?

1.2. Problem Definition

The game environment can be modeled as a Markov Decision Process (MDP):

$$M = (S, A, P, R, \gamma)$$

where:

- S: state space
- A: action space
- P: transition probability
- R: reward function
- γ: discount factor

At each time step t, the agent observes the current state s_t, selects an action a_t, and receives a reward r_t.

The objective is to learn a policy

$$\pi(a \mid s)$$

that maximizes the expected cumulative reward:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

For our platformer environment, the agent needs to learn how to move from the starting position toward the goal while avoiding falling from platforms.

A simplified action space can be defined as:

$$A = \{\text{Left, Right, Jump, None}\}$$

or combinations such as:

$$A = \{\text{Left, Right, Left+Jump, Right+Jump, None}\}$$

The main challenge is that the consequences of an action may not be immediately observable. For example, pressing the right key during a jump may affect the landing position several frames later.

1.3. Challenges of 2D Precision Platformers

There are several characteristics that make precision platformers interesting reinforcement learning environments.

1.3.1 Precise movement

The agent must control its position with relatively high precision. A small difference in horizontal position can determine whether the character lands on a platform or falls.

1.3.2 Temporal dependency

The correct action often depends on previous actions. For example: Move Right → Gain horizontal velocity → Jump → Continue Right → Release / change direction → Land on platform

Therefore, the agent cannot always make good decisions based solely on the current frame.

1.3.3 Sparse rewards

If the agent only receives a large reward after completing a level, it may take a very large number of attempts before discovering useful behavior.

This is a known difficulty in game-based reinforcement learning. OpenAI's Gym Retro documentation notes that sparse-reward games and tasks requiring planning over longer time horizons can be significantly harder for existing RL algorithms.

1.3.4 Exploration

A randomly initialized agent will frequently fall from platforms.

Consequently, the agent must balance:

- exploration of new actions
- exploitation of successful behaviors

1.3.5 Generalization

An agent that memorizes one level may not necessarily learn a general platform-navigation strategy.

Therefore, we may investigate whether an agent trained in several level layouts can perform on previously unseen layouts.

1.4. Existing Approaches

1.4.1 Deep Q-Network

DQN combines Q-learning with a deep neural network. Instead of maintaining a traditional Q-table, the neural network estimates the value of different actions.

Mnih et al. demonstrated that DQN could learn directly from high-dimensional visual observations and achieve strong performance across Atari games.

However, DQN is primarily designed for discrete action spaces and can become difficult to train when the environment requires fine-grained control.

For a precision platformer, repeatedly choosing discrete actions may result in less precise movement.

1.4.2 Proximal Policy Optimization

Proximal Policy Optimization (PPO) is a policy-gradient reinforcement learning algorithm proposed by Schulman et al. PPO optimizes a clipped surrogate objective to prevent excessively large policy updates.

PPO has also been applied to game environments and is supported by the Gym Retro framework.

Its relatively simple implementation and ability to handle stochastic policies make PPO a suitable candidate for our project.

Therefore, our initial implementation will focus on PPO.

1.5. Proposed Approach

Our proposed system consists of four major components: 2D Platformer Game、State / Observation、PPO RL Agent、Action.

The agent will repeatedly interact with the game and update its policy based on the collected experience.

2. Method

2.1. Environment

We will construct or use a simple 2D platformer environment with the following elements:

- Player character
- Platforms
- Gravity
- Horizontal movement
- Jumping
- Collision detection
- Goal position
- Death/failure condition

The environment will initially contain relatively simple levels so that the agent can learn basic movement before more complicated levels are introduced.

The difficulty can gradually increase by changing:

- platform distance
- platform width
- vertical difference
- number of platforms
- required movement precision

2.2. State Representation

We will initially investigate two types of state representations.

2.2.1 Game-State Representation

Instead of directly using raw images, the agent receives numerical information such as:

$$s_t = [x_p, y_p, v_x, v_y, x_g, y_g, x_1, y_1, w_1, \dots]$$

where:

- x_p, y_p: player position
- v_x, v_y: player velocity
- x_g, y_g: goal position
- x_i, y_i, w_i: platform information

This approach significantly reduces the dimensionality of the problem and allows us to focus on reinforcement learning rather than computer vision.

2.2.2 Visual Representation

As an extension, the agent may receive the game screen as input.

A CNN can extract spatial features from the game screen before the policy and value networks make predictions.

This approach is more challenging but closer to the setup used by DQN, where agents learn from visual observations.

2.3. Action Space

The initial action space will be discrete:

Action	Description
0	Do nothing
1	Move left
2	Move right
3	Jump
4	Left + Jump
5	Right + Jump

If the game automatically jumps when landing or uses another jumping mechanism, the action space can be simplified.

An important experimental variable is whether the agent should control **jumping explicitly** or whether jumping should be automatically triggered.

2.4. PPO Agent

We will use PPO as the main reinforcement learning algorithm.

At each time step:

1. Observe state s_t.
2. The policy network generates an action distribution.
3. Sample action a_t.
4. Execute the action in the environment.
5. Receive reward r_t.
6. Store the transition.
7. After collecting a batch of trajectories, update the policy.

The PPO objective uses the probability ratio:

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{old}}(a_t \mid s_t)}$$

and clips this ratio within a specified range:

$$L^{CLIP}(\theta) = E_t[\min(r_t, (\theta)A_t clip(r_t, (\theta), 1 - \epsilon 1 + \epsilon)A_t)]$$

where A_t represents the advantage estimate.

The clipping mechanism limits excessively large policy updates and helps improve training stability.

2.5. Reward Function

Reward design is expected to be one of the most important parts of this project.

A basic reward function could be:

$$R_t = w_p \Delta x + w_g R_{goal} - w_d R_{death} - w_s R_{stuck}$$

where:

- Δx: progress toward the goal
- R_{goal}: reward for reaching a new platform or goal
- R_{death}: penalty for dying
- R_{stuck}: penalty for remaining in the same area
- w_p, w_g, w_d, w_s: weighting parameters

For example:

Action	Reward
Move toward goal	+0.1
Reach a new platform	+1
Reach checkpoint	+5
Complete level	+20
Fall / die	-5
Remain stuck	-0.1

These values are preliminary and will be adjusted experimentally.

An important research issue is **reward shaping**. An improperly designed reward function can result in unintended behavior. For example, an agent may discover a way to repeatedly obtain small rewards without actually completing the intended objective. This phenomenon has been observed in game-based RL environments.

Therefore, we will compare different reward designs.

2.6. Curriculum Learning

Because the complete task may initially be too difficult, we plan to gradually increase the difficulty.

Stage 1:

Simple platforms.

Stage 2:

Different platform heights.

Stage 3:

Longer jumps and smaller platforms.

Stage 4:

Multiple platforms require sequential decisions.

Stage 5:

Previously unseen level layouts.

This allows us to investigate whether learning easier tasks first helps the agent solve more difficult environments.

2.7. Training Procedure

The training process will consist of several stages, beginning with a simple 2D platformer environment and gradually increasing the difficulty. We will use Proximal Policy Optimization (PPO) as the primary reinforcement learning algorithm.

First, the agent is initialized with randomly initialized policy and value networks. At the beginning of each episode, the game environment is reset and the player is placed at a predefined starting position. The agent then receives an observation describing the current game state, including the player's position, velocity, and the positions of nearby platforms.

At each time step, the policy network produces a probability distribution over the available actions. The agent selects an action according to this distribution and sends the action to the game environment. The environment then updates the player's position according to the game physics and returns the next observation and reward.

The interaction process is repeated until the agent reaches the goal, falls from the platforms, or reaches the maximum episode length.

The collected trajectories are stored in a rollout buffer. After enough time steps have been collected, the PPO algorithm calculates the return and advantage for each transition. The policy network and value network are then updated using the collected experience. This process is repeated for multiple training iterations.

The overall training procedure is:

1. Initialize the PPO policy and value networks.
2. Reset the platformer environment.
3. Observe the current state.
4. Select an action using the current policy.
5. Execute the action in the game environment.
6. Receive the next state and reward.
7. Store the transition in the rollout buffer.
8. Repeat Steps 3–7 until enough samples are collected or the episode terminates.
9. Calculate the advantage estimates and returns.
10. Update the policy and value networks using the PPO objective.
11. Repeat the training process for multiple iterations.
12. Periodically evaluate the agent without exploration and record its performance.

To improve training stability, we will initially use relatively simple levels with large platforms. Once the agent can reach the goal, the difficulty will gradually increase by reducing platform width, increasing the distance between platforms, and introducing larger vertical differences. This curriculum learning strategy is intended to allow the agent to learn basic movement before attempting more difficult platforming tasks.

During training, we will periodically save the agent's model and evaluate its performance on separate test levels. The evaluation results will be used to determine whether the learned policy can generalize beyond the levels encountered during training.

The overall training pipeline can be summarized as

follows:

Environment → Observation → PPO Policy → Action → Environment → Reward → Rollout Buffer → PPO Update → Evaluation

This procedure will be repeated until the agent reaches a stable performance level or the predefined training budget is exhausted.

2.8. Evaluation Metrics

The trained agent will be evaluated using several metrics:

2.8.1 Level Completion Rate

$$CompletionRate = \frac{\text{Successful Episodes}}{\text{Total Episodes}}$$

This measures how often the agent successfully completes a level.

2.8.2 Maximum Progress

The maximum horizontal or vertical progress achieved before death.

2.8.3. Number of Deaths

A lower death count indicates better survival.

2.8.4. Episode Reward

The cumulative reward obtained during each episode.

2.8.5. Training Efficiency

We will measure how many environment steps are required before the agent reaches a certain performance level.

2.8.6. Generalization

The agent will be evaluated at levels that were not included during training.

2.9. Expected Results

We expect the proposed reinforcement learning agent to gradually learn effective movement and navigation strategies through repeated interaction with the 2D precision platformer environment.

First, we expect the agent's performance to improve as the number of training steps increases. During the early stage of training, the agent is expected to move almost randomly and frequently fall from platforms. As training progresses, the agent should learn basic movement patterns, such as moving toward platforms, controlling its horizontal position during jumps, and adjusting its movement before landing.

We expect the average episode reward to increase and the death rate to decrease during training. The maximum progress achieved by the agent should also gradually increase as it learns to navigate through more platforms.

We also expect reward shaping to have a significant effect on the learning process. Compared with a sparse reward that only provides feedback when the agent reaches the final goal, intermediate rewards based on progress or reaching platforms may allow the agent to learn useful behaviors more efficiently.

In addition, we will compare agents trained under different conditions. For example, we will compare different reward functions and state representations to investigate their effects on learning performance. If time permits, we will also compare a state-based agent with an image-based agent that receives visual observations from the game screen.

The result will therefore not be evaluated only by whether the agent can complete a single level. Instead, we will analyze the learning curve, completion rate, movement behavior, reward design, and generalization performance to determine how effectively reinforcement learning can be applied to 2D precision platformer games.

4. References

[1] V. Mnih, K. Kavukcuoglu, D. Silver, et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, pp. 529–533, 2015.

[2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” arXiv:1707.06347, 2017.

[3] A.Nichol, V. Pfau, C. Hesse, O. Klimov, and J. Schulman, “Gotta Learn Fast: A New Benchmark for Generalization in RL,” arXiv:1804.03720, 2018.

[4] OpenAI, “Gym Retro,” 2018. Gym Retro provides a framework for using video games as reinforcement learning environments and studying generalization across games and levels.

[5] OpenAI, “Procgen Benchmark,” 2019. The benchmark investigates generalization in reinforcement learning using procedurally generated environments and evaluates agents on previously unseen levels.